

Estudio Comparativo de Heurísticas, Algoritmo Codicioso y A* en el Puzzle-8 y sus Generalizaciones

Asignatura: Inteligencia Artificial

Docente: Msc. Víctor Rodríguez Estévez

Integrantes del grupo: Nazaret Fabianne Claire Martinez, Igor Alfredo Pereira Rosado, Ariana Aylen Pita Vargas, Maria Jose Sandoval Orellana

Septiembre de 2026

Resumen

En este trabajo se implementan y comparan los algoritmos de búsqueda informada Codicioso (*Greedy Best-First Search*) y A* aplicados al Puzzle-8, junto con seis funciones heurísticas de distinto grado de información. Se verificó formalmente, sobre el espacio completo de 181.440 estados solubles, tanto la **admisibilidad** como la **consistencia** de cada heurística. Se realizó un experimento controlado sobre 1000 configuraciones iniciales aleatorias y solubles, combinando ambos algoritmos con ocho configuraciones de heurística ($1000 \times 8 \times 2 = 16.000$ corridas), registrando pasos, tiempo de cómputo, nodos expandidos, nodos generados y memoria máxima utilizada. Los resultados se analizaron mediante estadística descriptiva, la prueba no paramétrica de Friedman y el post-hoc de Nemenyi, determinando la heurística de *Pattern Database* (H5) como la de mejor desempeño global. Adicionalmente se generalizó el problema a tableros $N \times N$ y se determinó experimentalmente el límite práctico de escalabilidad de A* con la heurística de distancia Manhattan, ajustando un modelo de crecimiento potencial. Por último, se desarrolló un juego interactivo en PyGame que incorpora un asistente basado en A* capaz de resolver el rompecabezas en tiempo real.

Índice

1. Objetivos	2
2. Marco Teórico	2
2.1. El Puzzle-8	2
2.2. Algoritmos de Búsqueda Informada	2
2.2.1. Algoritmo Codicioso (Greedy Best-First Search)	2
2.2.2. Algoritmo A*	2
2.3. Funciones Heurísticas Propuestas	3
3. Verificación Formal de las Heurísticas	3
3.1. Admisibilidad	3
3.2. Consistencia	4
3.3. Hallazgo sobre H4	4
4. Entorno de Ejecución	5

5. Diseño Experimental	5
5.1. Experimento 1: Comparación de Heurísticas en Puzzle-8	5
5.2. Experimento 2: Escalabilidad por Tamaño de Tablero	5
6. Resultados del Experimento 1	6
6.1. Validación de la implementación	6
6.2. Estadísticas Descriptivas	6
6.3. Comparación de Heurísticas	9
6.4. Comparación entre Algoritmos	10
7. Resultados del Experimento 2: Escalabilidad	10
7.1. Tasa de éxito por tamaño de tablero	10
7.2. Modelado del Crecimiento	11
8. Juego Interactivo en PyGame	12
9. Limitaciones	14
10. Conclusión	14
Referencias	15

1. Objetivos

1. Implementar el algoritmo Codicioso (*Greedy Best-First Search*) y el algoritmo A* para resolver el Puzzle-8.
2. Diseñar y formular al menos cuatro funciones heurísticas admisibles y consistentes para el Puzzle-8.
3. Realizar un experimento controlado con 1000 distribuciones aleatorias solubles del Puzzle-8.
4. Medir y registrar, para cada combinación algoritmo-heurística: número de pasos de la solución, tiempo de ejecución, número de nodos expandidos, número de nodos generados y memoria máxima utilizada.
5. Aplicar pruebas estadísticas formales para determinar si existen diferencias significativas entre heurísticas y seleccionar la mejor.
6. Explorar la escalabilidad del enfoque variando el tamaño del tablero ($N \times N$, con $N = 3, 4, 5, \dots$) y determinar hasta qué tamaño es factible resolver con recursos computacionales razonables.

2. Marco Teórico

2.1. El Puzzle-8

El Puzzle-8 consiste en un tablero de 3×3 con 8 fichas numeradas del 1 al 8 y un espacio vacío (representado por 0). El objetivo es alcanzar la configuración meta:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}$$

Las acciones permitidas son mover el espacio vacío hacia arriba, abajo, izquierda o derecha, intercambiándolo con la ficha adyacente. El espacio de estados del Puzzle-8 tiene $9!/2 = 181.440$ configuraciones solubles (la mitad de las $9!$ permutaciones posibles, ya que la solubilidad depende de la paridad del número de inversiones).

2.2. Algoritmos de Búsqueda Informada

2.2.1. Algoritmo Codicioso (Greedy Best-First Search)

Expande el nodo con el menor valor de $h(n)$, sin considerar el costo acumulado $g(n)$. No es óptimo ni completo en espacios infinitos, pero suele ser muy rápido.

2.2.2. Algoritmo A*

Expande el nodo con el menor valor de $f(n) = g(n) + h(n)$. Si $h(n)$ es admisible (nunca sobreestima el costo real) y consistente, A* es óptimo y completo.

Ambos algoritmos se implementaron con una cola de prioridad (**heapq**) sobre la clase **AgenteBuscador** del curso, agregando un contador de desempate (los caminos, al ser listas de tableros, no son comparables entre sí cuando dos nodos empatan en f) y un conjunto de visitados sobre tuplas para que la búsqueda de duplicados sea $O(1)$ en vez de $O(n)$.

Nota sobre el framework base. La implementación original de `AgenteBuscador.programa()` (amplitud, profundidad, costo uniforme) no filtra los valores `None` que devuelve `generar_hijos` ante movimientos inválidos —se verificó en el código fuente que la condición `if hijo not in visitados` no distingue `None` de un estado real—, lo cual provoca un error si se intentan usar esas técnicas originales directamente sobre `AgenteRK8`. Las técnicas nuevas (`Codicioso`, `A*`) filtran explícitamente estos valores.

2.3. Funciones Heurísticas Propuestas

Se implementaron seis funciones heurísticas sobre el agente `AgenteRK8`, con distinto grado de información:

H1 — Fichas mal colocadas (Hamming). Cuenta el número de fichas (sin contar el blanco) que no están en su posición meta. Es admisible pero débil.

H2 — Distancia Manhattan. Suma, para cada ficha, la distancia $|\Delta x| + |\Delta y|$ entre su posición actual y su posición meta. Es admisible y consistente.

H3 — Manhattan + conflicto lineal. $h_3 = h_2 + 2 \cdot C(s)$, donde $C(s)$ es el número mínimo de fichas que deben abandonar una fila o columna para eliminar los conflictos lineales (pares de fichas correctamente alineadas pero en orden invertido). Se calcula mediante un algoritmo goloso sobre el grafo de conflictos de cada línea, que para líneas de tamaño 3 (único caso relevante en 3×3) coincide con el óptimo.

H4 — Manhattan + conflicto lineal + penalización de esquinas. $h_4 = h_3 + E(s)$, donde $E(s)$ penaliza fichas de esquina mal ubicadas cuando el espacio vacío no está en la esquina opuesta.

H5 — Pattern Database. Se particionan las fichas en dos grupos disjuntos, $\{1, 2, 3, 4\}$ y $\{5, 6, 7, 8\}$, y se precalcula mediante BFS 0–1 la distancia exacta de cada patrón abstraído a su meta, ignorando las fichas del otro grupo. La heurística final es la suma de ambas distancias.

H6 — Peso variable. $h_6 = w \cdot h_2 + (1 - w) \cdot h_1$, con $w \in [0, 1]$. Se evaluaron los pesos $w = 0, 3$, $w = 0, 6$ y $w = 1, 0$.

3. Verificación Formal de las Heurísticas

3.1. Admisibilidad

En vez de verificar la admisibilidad con ejemplos sueltos, se implementó un script (`verificacion/verificar_adm`) que recorre mediante BFS desde el estado meta las 181.440 configuraciones solubles del Puzzle-8, calcula la distancia óptima real para cada una, y compara ese valor contra lo que reporta cada heurística. Una heurística es admisible si nunca sobreestima ese valor en ninguna de las 181.440 configuraciones.

Cuadro 1: Verificación exhaustiva de admisibilidad (181.440 estados).

Heurística	¿Admisible?	Fallas	Peor exceso
H1	Sí	0 / 181.440	—
H2	Sí	0 / 181.440	—
H3	Sí	0 / 181.440	—
H4	No	1.141 / 181.440	2
H5	Sí	0 / 181.440	—
H6 ($w = 0,3, 0,6, 1,0$)	Sí	0 / 181.440	—

3.2. Consistencia

La admisibilidad es una condición *global* (compara $h(s)$ contra la distancia óptima final). La **consistencia** es una condición *local*, más fuerte, que exige que para todo par estado-vecino (s, s') conectado por un movimiento válido de costo 1:

$$h(s) \leq 1 + h(s')$$

Se implementó un segundo script (`verificacion/verificar_consistencia.py`) que reconstruye el mismo espacio de estados y revisa esta desigualdad sobre las 483.840 aristas del grafo de estados (cada uno de los 181.440 estados solubles tiene, en promedio, algo menos de 3 vecinos válidos).

Cuadro 2: Verificación exhaustiva de consistencia (483.840 aristas estado-vecino).

Heurística	¿Consistente?	Fallas	Peor exceso
H1	Sí	0 / 483.840	—
H2	Sí	0 / 483.840	—
H3	Sí	0 / 483.840	—
H4	No	41.712 / 483.840 (8,6 %)	2
H5	Sí	0 / 483.840	—

H6 no se verificó por separado con el script: al ser una combinación convexa $h_6 = w \cdot h_2 + (1 - w) \cdot h_1$ de dos heurísticas consistentes, hereda la propiedad para cualquier $w \in [0, 1]$ — si $h_2(s) \leq 1 + h_2(s')$ y $h_1(s) \leq 1 + h_1(s')$, entonces su combinación convexa también cumple la desigualdad, ya que $w + (1 - w) = 1$.

3.3. Hallazgo sobre H4

H4 (Manhattan + conflicto lineal + penalización de esquinas) **no resultó admisible ni consistente**. La consigna original pedía explícitamente “diseñar $E(s)$ de modo que preserve la admisibilidad”, lo cual sugiere que se trata de una condición deliberadamente difícil de lograr. Se decidió conservar H4 tal como fue diseñada y documentar el hallazgo, en lugar de forzar una corrección artificial: el penalizador de esquinas es un caso clásicamente delicado, donde es fácil sobreestimar el costo real cuando el espacio vacío está cerca de la esquina en cuestión.

Esto implica que H4 puede usarse en el algoritmo Codicioso (que no requiere admisibilidad), pero no garantiza la optimalidad de A* cuando se usa con ella. A pesar de ello, como se detalla en la Sección 6.1, H4 no produjo caminos subóptimos en ninguna de las 1000 instancias del experimento — lo cual no es una garantía general, sino una observación empírica sobre esta muestra particular.

4. Entorno de Ejecución

Los tiempos reportados en este informe dependen del equipo utilizado. El Experimento 1 y el Experimento 2 se ejecutaron en el siguiente equipo:

Cuadro 3: Especificaciones del equipo utilizado para los experimentos.

Componente	Detalle
Equipo	Lenovo IdeaPad 3 15ITL6 (modelo 82H8)
Procesador	Intel® Core™ i7-1165G7 (11. ^a generación), 2,80 GHz
Memoria RAM instalada	20,0 GB (19,8 GB utilizables)
Sistema operativo	Microsoft Windows 11 Home (compilación 26200)

5. Diseño Experimental

5.1. Experimento 1: Comparación de Heurísticas en Puzzle-8

Generación de instancias. Se generaron 1000 configuraciones iniciales aleatorias y únicas, garantizando solubilidad mediante el chequeo de paridad de inversiones (una permutación de las 8 fichas es soluble si y solo si su número de inversiones es par).

Algoritmos y heurísticas. Se combinaron los algoritmos Codicioso y A* con las heurísticas H1, H2, H3, H4, H5 y H6 (con $w = 0,3, 0,6$ y $1,0$, tratadas como configuraciones independientes), dando un total de 8 configuraciones de heurística $\times$ 2 algoritmos $\times$ 1000 instancias = 16.000 corridas.

Métricas registradas. Por cada corrida: si se encontró solución, número de pasos, tiempo de ejecución (ms), nodos expandidos, nodos generados y memoria máxima (definida como el máximo de `len(frontera) + len(visitados)` durante la búsqueda). No se impuso ningún límite de tiempo ni de nodos en este experimento: el espacio de estados del Puzzle-8 (181.440 configuraciones) es lo suficientemente chico como para que las 16.000 corridas terminen sin necesidad de un corte artificial.

5.2. Experimento 2: Escalabilidad por Tamaño de Tablero

Se generalizó el Puzzle-8 a tableros $N \times N$ (**AgenteRKN**), generalizando las heurísticas H1 y H2 — ambas siguen siendo admisibles y consistentes para cualquier N , por el mismo argumento de la Sección 3 aplicado a tableros de tamaño arbitrario—. H5 (Pattern Database) no se generalizó a N arbitrario, ya que requeriría rediseñar la partición de grupos de fichas para cada tamaño de tablero, lo cual excede el alcance de esta sección y se deja como posible extensión futura. Por esa razón, el Experimento 2 usa A* con **H2 (Manhattan)**, la heurística admisible y consistente de generalización directa más fuerte disponible sin ese rediseño — no la heurística de mejor desempeño global del Experimento 1 (H5), que no generaliza sin ese trabajo adicional.

Generación de instancias. Para $N = 3$ se usó el mismo generador con chequeo de paridad. Para $N \geq 4$, en lugar de permutaciones 100 % aleatorias (que pueden generar instancias intratables, muy lejos de la meta), se generaron instancias mediante una caminata aleatoria de longitud controlada desde el estado meta (40 movimientos para $N = 4$, 60 para $N = 5$, 80 para $N = 6$) — alternativa explícitamente sugerida en el enunciado de la práctica —, evitando deshacer el movimiento inmediatamente anterior.

Protección de recursos. Se impuso un presupuesto de 300.000 nodos expandidos y 60 segundos por instancia, con 30 instancias por tamaño de tablero. Sin este límite, una instancia difícil en $N \geq 4$ puede agotar la memoria disponible del sistema.

6. Resultados del Experimento 1

6.1. Validación de la implementación

Antes de interpretar cualquier resultado estadístico, se verificó la corrección de la implementación: en las 1000 instancias, A* encontró **exactamente el mismo número de pasos** de solución sin importar la heurística utilizada (0 inconsistencias detectadas sobre las 1000 instancias, verificado directamente sobre `resultados_experimento1.csv`). Esta es una validación fuerte de que la implementación es correcta: un error en el código de búsqueda, o en alguna heurística que se creyera admisible sin serlo, se manifestaría como una discrepancia en el óptimo encontrado por A* entre distintas heurísticas. El algoritmo Codicioso, por su parte, encontró solución en el 100 % de las 8.000 corridas que le correspondieron.

6.2. Estadísticas Descriptivas

La Tabla 4 resume la mediana, media y desviación estándar de nodos expandidos por A*, para cada heurística. Se observa una relación monótona entre el grado de información de la heurística y la cantidad de nodos expandidos: H5 es sistemáticamente la más eficiente, mientras que H1 es la más costosa, con una dispersión mucho mayor.

Cuadro 4: Nodos expandidos por A*, según heurística (1000 instancias).

Heurística	Mediana	Media	Desv. estándar
h1	9.411,5	15.884,3	17.381,6
h2	933,0	1.580,8	1.907,3
h3	512,0	887,0	1.063,1
h4	374,5	654,1	774,1
h5	89,0	138,0	152,4
h6_w0.3	3.478,0	6.289,7	7.603,8
h6_w0.6	1.670,5	2.967,5	3.678,7
h6_w1.0	933,0	1.580,8	1.907,3

Chequeo de sanidad: h6_w1.0 y h2 arrojan valores idénticos en todas las estadísticas. Esto es matemáticamente esperado, ya que con $w = 1,0$ la heurística H6 se reduce exactamente a H2, y confirma que la implementación de H6 es correcta.

La Tabla 5 muestra la misma comparación para el tiempo de ejecución (ms), que sigue el mismo patrón que los nodos expandidos.

Cuadro 5: Tiempo de ejecución en ms por A*, según heurística (1000 instancias).

Heurística	Mediana	Media	Desv. estándar
h1	593,8	1.055,7	1.285,2
h2	55,8	98,7	134,7
h3	43,0	78,9	103,0
h4	35,5	64,8	84,0
h5	5,6	9,3	12,3
h6_w0.3	238,8	461,7	835,8
h6_w0.6	105,2	199,9	262,6
h6_w1.0	54,5	102,5	133,2

Las Figuras 1 y 2 muestran la distribución completa (sin valores atípicos) de nodos expandidos para ambos algoritmos.

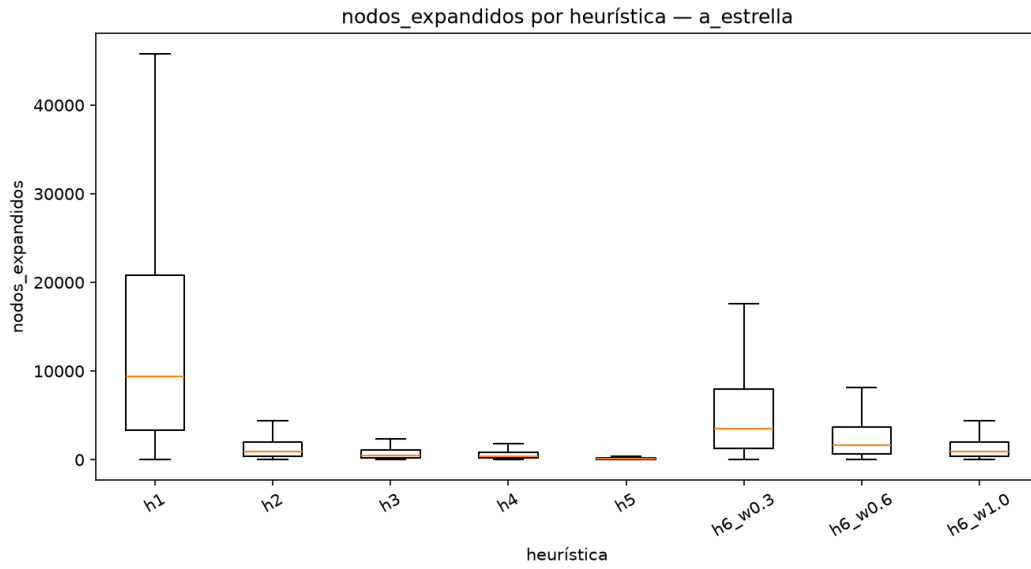


Figura 1: Distribución de nodos expandidos por heurística, algoritmo A* (sin valores atípicos).

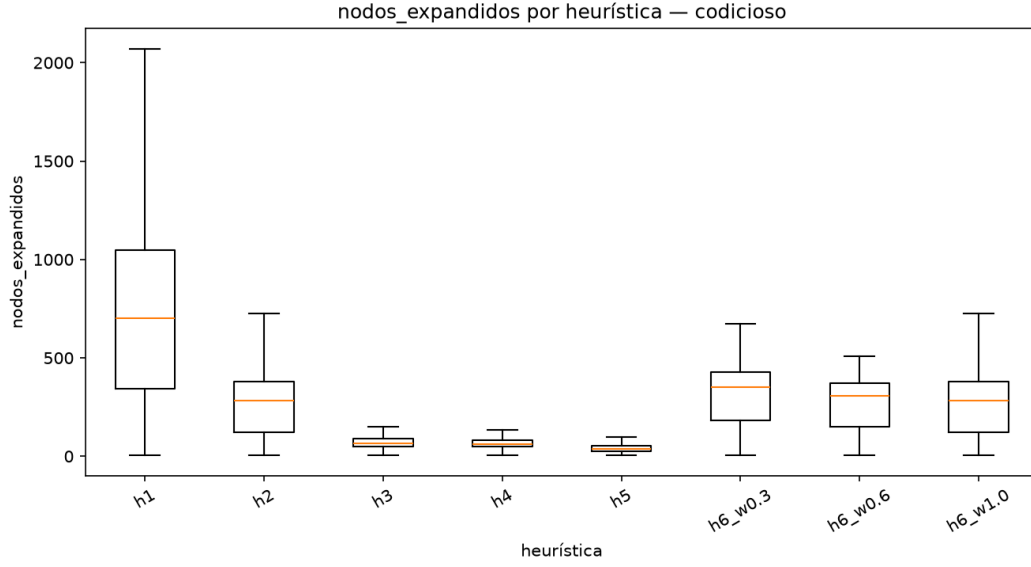


Figura 2: Distribución de nodos expandidos por heurística, algoritmo Codicioso (sin valores atípicos).

Como complemento a los boxplots, las Figuras 3 y 4 presentan la media de nodos expandidos junto con su intervalo de confianza al 95 % (calculado mediante la distribución t de Student, $IC_{95\%} = \bar{x} \pm t_{0,975, n-1} \cdot \frac{s}{\sqrt{n}}$).

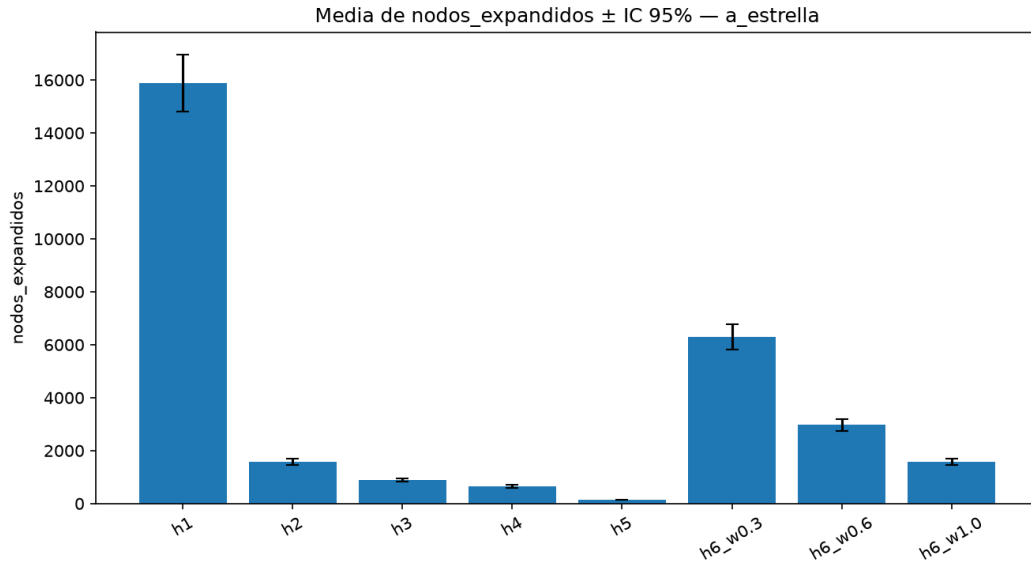


Figura 3: Media de nodos expandidos $\pm$ intervalo de confianza al 95 %, algoritmo A*.

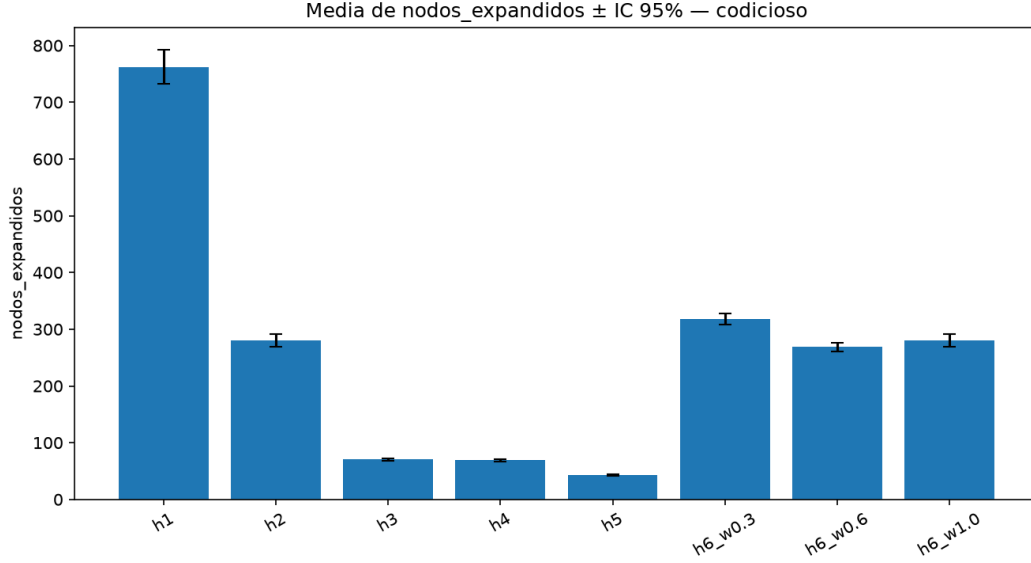


Figura 4: Media de nodos expandidos $\pm$ intervalo de confianza al 95 %, algoritmo Codicioso.

Los boxplots de tiempo de ejecución (`boxplot_tiempo_a_estrella.png` y `boxplot_tiempo_codicioso.png`, incluidos en el repositorio de código en `experimentos/resultados/`) muestran el mismo patrón relativo entre heurísticas que la Tabla 5.

6.3. Comparación de Heurísticas

Prueba de Friedman. Dado que las 1000 instancias reciben las 8 configuraciones de heurística (diseño de bloques completos), se aplicó la prueba no paramétrica de Friedman sobre nodos expandidos y tiempo de ejecución, para cada algoritmo por separado:

Cuadro 6: Resultados de la prueba de Friedman.

Algoritmo	Métrica	χ_F^2	p	Resultado
Codicioso	Nodos expandidos	5.282,48	$p < 0,001$	Significativo
Codicioso	Tiempo (ms)	4.030,11	$p < 0,001$	Significativo
A*	Nodos expandidos	6.968,58	$p < 0,001$	Significativo
A*	Tiempo (ms)	6.132,05	$p < 0,001$	Significativo

En los cuatro casos, la diferencia entre heurísticas es estadísticamente significativa. Con $n = 1000$ instancias esto era esperable incluso ante diferencias pequeñas; el análisis relevante para elegir la mejor heurística está en el post-hoc y el ranking compuesto, no en la significancia global de Friedman.

Post-hoc (Nemenyi). Al ser Friedman significativo en los cuatro casos, se aplicó la prueba post-hoc de Nemenyi (`scikit-posthocs`) para determinar qué pares de heurísticas difieren entre sí; las cuatro matrices completas de p -valores se guardan en `experimentos/resultados/posthoc_nemenyi_*.csv`. Como chequeo de sanidad adicional, la comparación entre h2 y h6_w1.0 arrojó un p -valor de 1,0 exacto — el resultado correcto, dado que ambas heurísticas son matemáticamente idénticas cuando $w = 1,0$.

Ranking compuesto. Para cada algoritmo, se rankearon las heurísticas según la mediana de

nodos expandidos, tiempo de ejecución y memoria máxima, promediando los tres rangos (rango 1 = mejor):

Cuadro 7: Ranking compuesto de heurísticas por algoritmo.

Algoritmo	Heurística	Rango promedio
A*	h5	1,000
A*	h4	2,000
A*	h3	3,000
A*	h6_w1.0	4,333
A*	h2	4,667
A*	h6_w0.6	6,000
A*	h6_w0.3	7,000
A*	h1	8,000
Codicioso	h5	1,000
Codicioso	h4	2,333
Codicioso	h3	2,667
Codicioso	h2	4,333
Codicioso	h6_w1.0	4,667
Codicioso	h6_w0.6	6,000
Codicioso	h6_w0.3	7,000
Codicioso	h1	8,000

Conclusión: H5 (Pattern Database) es la heurística de mejor desempeño global, en ambos algoritmos, de forma consistente en las tres métricas evaluadas. Es también, junto con H1, H2 y H3, una de las heurísticas formalmente admisibles y consistentes (Sección 3), por lo que su elección no compromete la optimalidad de A*.

6.4. Comparación entre Algoritmos

Codicioso es sistemáticamente más rápido que A* en nodos expandidos (por ejemplo, con H5 la mediana es 36 nodos para Codicioso frente a 89 para A*), pero **no garantiza optimalidad**: con H5, la mediana de pasos de Codicioso es 26 (media 27,47), frente a una mediana de 22 pasos para A* con cualquier heurística admisible — es decir, en la muestra de 1000 instancias, Codicioso con H5 encuentra soluciones en promedio aproximadamente un 18 % más largas que el óptimo. Este comportamiento es exactamente el esperado según la teoría: Codicioso prioriza velocidad sobre optimalidad al ignorar el costo acumulado $g(n)$.

7. Resultados del Experimento 2: Escalabilidad

7.1. Tasa de éxito por tamaño de tablero

Con 30 instancias por tamaño de tablero, dentro del presupuesto de 300.000 nodos / 60 segundos:

Cuadro 8: Tasa de éxito de $A^* + H2$ por tamaño de tablero.

N	Tasa de éxito	Interpretación
3	100 % (30/30)	$A^* + \text{Manhattan}$ resuelve todas las instancias sin dificultad
4	100 % (30/30)	Sigue siendo completamente factible
5	50 % (15/30)	Punto de quiebre: la mitad de las instancias exceden el presupuesto
6	3 % (1/30)	Prácticamente intratable con $A^* + \text{Manhattan}$ puro

N **máximo factible (con $\geq 50\%$ de éxito):** $N = 5$.

7.2. Modelado del Crecimiento

Con los datos de éxito de la tabla anterior, se ajustaron dos modelos de crecimiento sobre la mediana de nodos expandidos en función de N , mediante regresión lineal sobre los logaritmos:

Cuadro 9: Modelos de crecimiento ajustados (excluyendo $N = 6$, ver nota).

Modelo	Ecuación ajustada	R^2
Exponencial	$\text{nodos}(N) \approx 2,386 \cdot 6,978^N$	0,8822
Potencial	$\text{nodos}(N) \approx 0,161 \cdot N^{7,668}$	0,9496

El modelo **potencial** ofrece el mejor ajuste ($R^2 = 0,9496$).

Nota metodológica: sesgo de supervivencia. El punto $N = 6$ se excluyó deliberadamente del ajuste. Con solo 3 % de instancias resueltas a tiempo en ese tamaño, las pocas que sí terminaron son casi con certeza las más fáciles de esa muestra (menor número de nodos), lo que produce una mediana artificialmente baja y no representativa del costo real de resolver un tablero de 6×6 . Al incluir ese punto sesgado, el ajuste potencial cae a $R^2 = 0,52$; al excluirlo, sube a $R^2 = 0,95$. Este es un ejemplo concreto de *sesgo de supervivencia* (*survivorship bias*): medir únicamente sobre los casos que “sobrevivieron” al límite de tiempo/nodos infla artificialmente el desempeño aparente del tamaño más difícil.

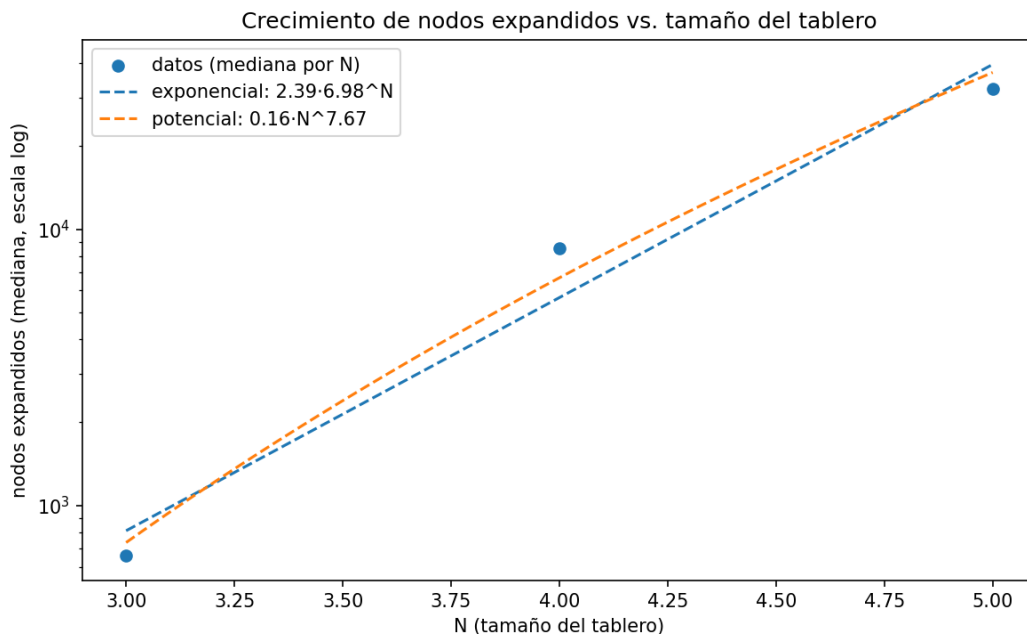


Figura 5: Ajuste de los modelos de crecimiento sobre $N = 3, 4, 5$ (escala logarítmica en el eje y). El punto $N = 6$ se excluyó del ajuste por sesgo de supervivencia.

8. Juego Interactivo en PyGame

Se desarrolló un juego jugable en PyGame (`juego/juego_puzzle.py`) que reutiliza exactamente el mismo código de búsqueda de las secciones anteriores (`busqueda/rkn_informado.py`).

Mecánica. El tamaño del tablero funciona como nivel de dificultad (Fácil 3×3 , Medio 4×4 , Difícil 5×5). El jugador mueve fichas haciendo clic sobre una celda adyacente al espacio vacío; la dirección del movimiento queda determinada por la ficha elegida, ya que cada ficha adyacente al vacío solo admite un movimiento posible. El tablero se genera siempre soluble, mediante una caminata aleatoria desde la meta.

Asistente (A*). Un botón dedicado invoca A* con la heurística Manhattan (H2) sobre el estado actual del tablero, y anima la solución encontrada paso a paso. La búsqueda del asistente está protegida con el mismo mecanismo de límite de nodos y tiempo utilizado en el Experimento 2 (con valores propios para uso interactivo: 400.000 nodos o 35 segundos). Si el tablero está muy revuelto y se alcanza ese límite, el juego lo indica explícitamente distinguiendo si fue el límite de nodos o el de tiempo el que se alcanzó — nunca se reporta como “sin solución”, ya que todo estado alcanzable jugando es soluble por construcción (los movimientos preservan la solubilidad).

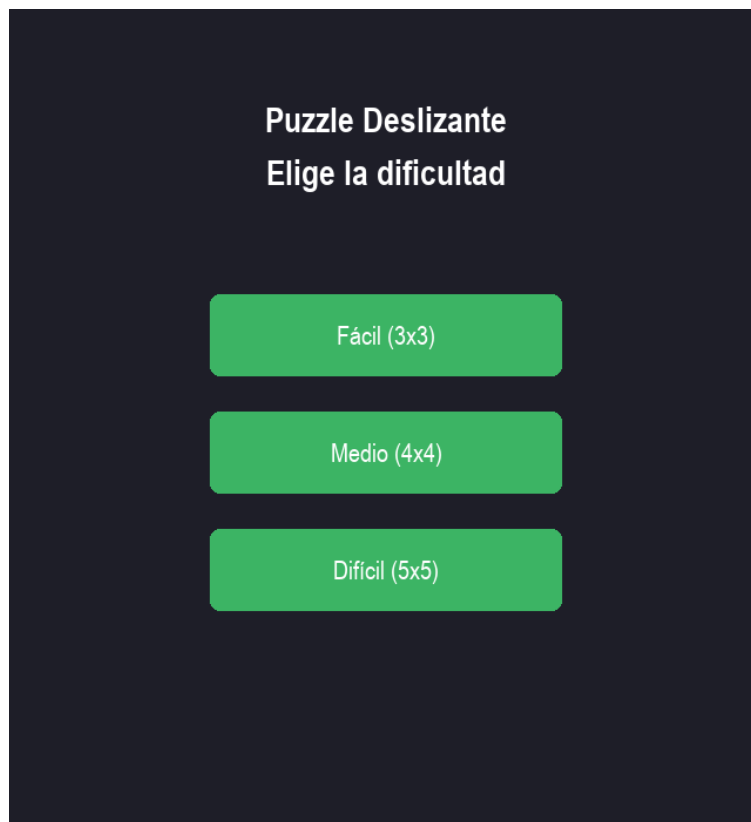


Figura 6: Pantalla de selección de dificultad del juego.

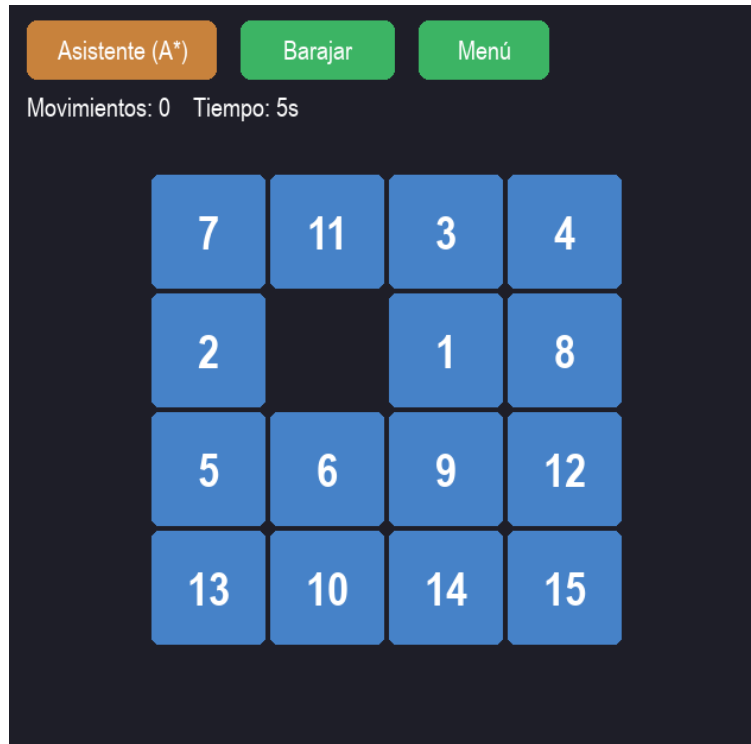


Figura 7: Tablero de juego en curso.

9. Limitaciones

- H4 no es admisible ni consistente (Sección 3); su buen desempeño empírico con A* en las 1000 instancias del Experimento 1 no es una garantía general de optimalidad.
- H5 (Pattern Database), la heurística de mejor desempeño global, no se generalizó a tableros $N \times N$ arbitrarios; el Experimento 2 usa H2 en su lugar por esa razón.
- El método de conflicto lineal usado en H3 (remoción golosa del vértice de mayor grado) da el resultado óptimo para líneas de tamaño 3, pero no está garantizado que sea óptimo para líneas más largas si se generalizara H3 a $N \times N$.
- El Experimento 2 se corrió con 30 instancias por tamaño y un límite de 60 segundos; con más instancias o un límite mayor, la tasa de éxito estimada para $N = 5$ (exactamente 50 % con esta muestra) podría variar.

10. Conclusión

Este trabajo permitió comprender de manera empírica y formal la importancia de las funciones heurísticas en la búsqueda informada. La verificación exhaustiva de admisibilidad y **consistencia** sobre el espacio de estados completo del Puzzle-8 (181.440 configuraciones, 483.840 aristas) permitió detectar que la heurística H4, diseñada según el criterio propuesto en la consigna, no cumple ninguna de las dos propiedades en todos los casos — un resultado legítimo que ilustra lo delicado que es diseñar penalizadores heurísticos sin sobreestimar el costo real.

El experimento controlado sobre 1000 instancias, analizado mediante la prueba de Friedman y el post-hoc de Nemenyi, determinó que la heurística de Pattern Database (H5) ofrece el mejor balance entre optimalidad, tiempo, nodos expandidos y memoria, tanto en Codicioso como en A*. La exploración de escalabilidad sobre tableros $N \times N$ mostró que, con una heurística de Manhattan simple, el límite práctico de A* se encuentra en torno a $N = 5$, y que el crecimiento del costo computacional sigue aproximadamente un modelo potencial ($R^2 = 0,95$) en función del tamaño del tablero.

Finalmente, el desarrollo del juego interactivo permitió trasladar estos conceptos a una aplicación tangible, donde el mismo algoritmo A* que se analizó estadísticamente puede observarse resolviendo el rompecabezas en tiempo real frente al usuario.

Referencias

Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson. (Definiciones de admisibilidad, consistencia, y optimalidad de A* y Greedy Best-First Search.)